Journal of Machine Learning Research 27 (2026) 1-15 Submitted 4/26; Published

GraFlag: Reproducible Distributed Benchmarking for Graph Anomaly Detection

Ahmed Youcef Benhalima AHMED-YOUCEF.BENHALIMA@ENSMA.FR
LIAS – ISAE-ENSMA, Poitiers, France

Seif-Eddine Benkabou SEIF.EDDINE.BENKABOU@UNIV-POITIERS.FR
LIAS – Université de Poitiers, France

Amin Mesmoudi AMIN.MESMOUDI@UNIV-POITIERS.FR
LIAS – Université de Poitiers, France

Allel Hadjali ALLEL.HADJALI@ENSMA.FR
LIAS – ISAE-ENSMA, Poitiers, France

Editor: TBD

Abstract

GraFlag is an open-source Python framework for *reproducible* benchmarking of graph anomaly detection (GAD) methods on distributed infrastructure. Although libraries like PyGOD provide algorithms and benchmarks like BOND define protocols, no integrated platform combines containerized execution, standardized results, and automated evaluation across distributed GPU clusters—leaving reproducibility fragile. GraFlag closes this gap with a modular architecture on Docker Swarm with NFS-based shared storage, so every experiment replays from a single command on a clean checkout. Nine standardized result types make outputs directly comparable across methods and re-runs, covering static and dynamic graphs at node, edge, and graph granularity. A CLI and web dashboard expose experiment management, real-time monitoring, and automatic generation of metrics and publication-ready plots. Available at `https://gbay7.github.io/graflag/`; installable via `pip install graflag`.

Keywords: Graph Anomaly Detection, Distributed Benchmarking, Docker Swarm, Reproducibility, Graph Neural Networks

1 Introduction

Graph anomaly detection (GAD) aims to identify unusual patterns in graph-structured data, with applications in fraud detection, network security, social network analysis, and biological networks (Ma et al., 2021). The field has seen rapid growth, with methods spanning diverse paradigms including autoencoders (Ding et al., 2019; Fan et al., 2020), contrastive learning (Xu et al., 2022; Liu et al., 2021), generative adversarial networks (Chen et al., 2020), and temporal approaches for dynamic graphs (Liu et al., 2023; Zheng et al., 2019).

Several tools have been developed to support GAD research. PyGOD (Liu et al., 2024) provides a Python library of detection algorithms with a unified API, while BOND (Liu et al., 2022) and GADBench (Tang et al., 2023) define standardized evaluation protocols for unsupervised and supervised settings, respectively. In the related domain of time series

©2026 Ahmed Youcef Benhalima, Seif-Eddine Benkabou, Amin Mesmoudi and Allel Hadjali.

License: CC-BY 4.0, see `https://creativecommons.org/licenses/by/4.0/`. Attribution requirements are provided at `http://jmlr.org/papers/v27/26-0000.html`.

anomaly detection, TimeEval (Wenig et al., 2022) demonstrates the value of distributed benchmarking toolkits. For tabular data, PyOD (Zhao et al., 2019) has become a widely adopted library. However, none of these tools provide an integrated platform that combines containerized execution environments, distributed GPU scheduling, standardized result formats across different graph types, and automated evaluation with visualization, leaving reproducibility of reported GAD benchmarks dependent on ad-hoc scripts, manual dataset downloads, and unpinned local environments.

To bridge this gap, we present GraFlag (**Gra**ph Anomaly **Flag**), a distributed benchmarking framework designed for end-to-end reproducibility, with four contributions: (*i*) containerized execution via Docker Swarm with automatic GPU allocation across a multi-node cluster, guaranteeing identical runtime environments across machines and re-runs; (*ii*) nine standardized result types covering three granularities (node, edge, graph) across three temporal modes (static, temporal, streaming), so any reported experiment can be replayed and compared from a single authoritative artifact; (*iii*) a plugin-based evaluation system that computes standard metrics and generates publication-ready visualizations directly from that artifact; and (*iv*) a CLI and web dashboard with real-time experiment monitoring, complemented by on-demand dataset fetching from original sources via versioned per-dataset metadata.

2 Framework Design and Implementation

GraFlag is a lightweight Python client (Python 3.10+, depending only on `docker`, `pyyaml`, `flask`, and `flask-socketio`) that orchestrates experiments on a remote GPU cluster. The client itself needs no GPU; all computation is offloaded.

Architecture. The framework adopts a three-layer architecture (Figure 1). The *Client Layer* provides the CLI, Python API, and web dashboard, communicating with the cluster via SSH tunnels. The *Cluster Layer* is a Docker Swarm comprising a manager node with a local image registry and GPU-enabled worker nodes. The *Shared Storage Layer* is an NFS-mounted directory organized into `methods/`, `datasets/`, and `experiments/` repositories, accessible from all nodes.

Method Integration. To run on this infrastructure, each detection method is packaged as a Docker image. GraFlag provides two integration patterns. For methods available in the PyGOD library (Liu et al., 2024), the `graflag_bond` adapter handles data loading, training, and result writing—requiring only a `.env` configuration and a `Dockerfile`. For other methods, a custom script uses the `ResultWriter` API to produce standardized outputs (Code Demo 1). Table 1 lists the currently integrated methods.

Standardized Result Types. Regardless of the integration pattern, every method must produce results in one of nine standardized formats, defined as the Cartesian product of three granularities—node, edge, and graph—and three temporal modes: *static* (1D score vectors), *temporal* (2D matrices indexed by entities $\times$ timestamps), and *streaming* (sparse 1D scores over a continuous timeline). The same nine-way classification drives the evaluator's metric dispatch.

Evaluation. Once an experiment completes, the `graflag_evaluator` computes AUC-ROC, AUC-PR, F1@K, Precision@K, and Recall@K from the standardized results, and generates publication-ready plots. Custom metrics can be added via a plugin system:

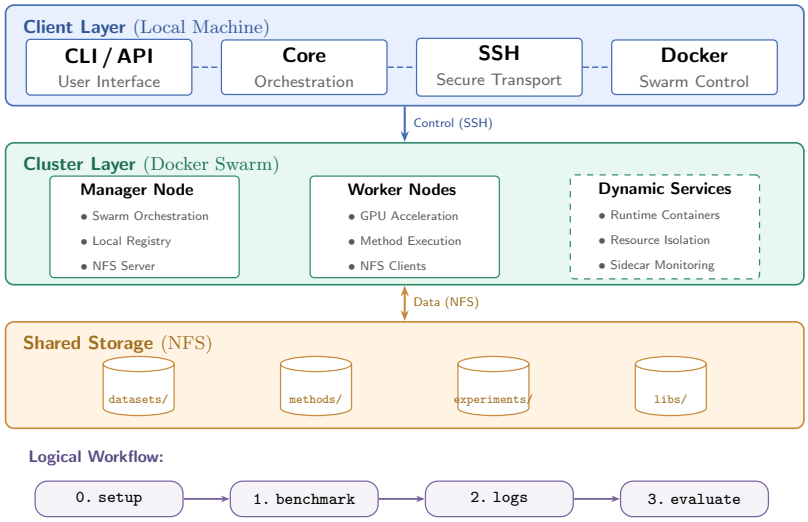

Figure 1: GraFlag architecture. The client orchestrates experiments remotely via SSH. Docker Swarm schedules method containers on GPU-enabled workers. All data flows through NFS-mounted shared storage.

Table 1: Detection methods in GraFlag. PyGOD methods (Liu et al., 2024) are integrated via `graflag_bond`; custom methods use `ResultWriter`. Additional methods are continuously being integrated.

Method	Graph	Backbone	GPU	Reference
PyGOD methods (DOMINANT, CONAD, AnomalyDAE, CoLA, GAAN, GAE, SCAN, etc.)	Static	Various	Yes/No	Liu et al. (2024)
TADDY	Dynamic	Transformer	Yes	Liu et al. (2023)
AddGraph	Dynamic	GCN	Yes	Zheng et al. (2019)
DynWalk	Dynamic	Embedding	No	Yu et al. (2018)
StrGNN	Dynamic	GNN	Yes	Cai et al. (2021)
GeneralDyG	Dynamic	GNN	Yes	Yang et al. (2025)
GADY	Dynamic	GNN	Yes	Lou et al. (2023)
SLADE	Dynamic	GNN	Yes	Lee et al. (2024)
AnoGraph	Dynamic	Sketch	No	Bhatia et al. (2023)
StreamSpot	Dynamic	Sketch	No	Manzoor et al. (2016)

`register_metric()` extracts the function source and persists it as a plugin file on the shared storage, so that the containerized evaluator loads it at runtime (Code Demo 2).

Workflow, Interfaces, and Reproducibility. The entire pipeline—from cluster setup through execution to evaluation—is exposed via a CLI (`graflag setup`, `graflag run`, `graflag evaluate`), the Python API shown above, and a web dashboard (`graflag gui`) with live status badges, log streaming, and one-click evaluation. Every experiment persists its full configuration, so any past run replays with `graflag run --from-config`.

Code Demo 1: Integrating a custom detection method.

```python
from graflag_runner import ResultWriter
writer = ResultWriter()
writer.spot("training", epoch=e, loss=l)
writer.save_scores(
    result_type="EDGE_STREAM_ANOMALY_SCORES",
    scores=scores, ground_truth=ground_truth)
writer.finalize()
```

Code Demo 2: Programmatic batch benchmarking via the Python API.

```python
from graflag import GraFlag
# Register a custom metric (persisted as plugin file)
def hits_at_100(scores, ground_truth, **kw):
    top = scores.argsort()[-100:]
    return {"hits@100": ground_truth[top].mean()}
gf = GraFlag()
gf.register_metric("EDGE_STREAM_ANOMALY_SCORES", hits_at_100)
# Run all method/dataset combinations
for m in gf.list_methods():
    for d in gf.list_datasets():
        exp = gf.run(m.name, d.name, build=True)
        gf.evaluate(exp)
        res = gf.get_evaluation_results(exp)
        print(f"{m.name}/{d.name}: AUC={res.metrics['auc_roc']:.3f}")
```

3 Quality and Accessibility

GraFlag is released under the MIT license and installable via `pip install graflag`. It includes a virtual development cluster (`graflag devcluster`) for local testing without physical infrastructure.

4 Conclusion and Future Plans

GraFlag provides containerized execution, standardized results, and automated evaluation for graph anomaly detection on Docker Swarm. Future work includes Apptainer/Singularity for HPC, statistical comparison tools, and hyperparameter tuning.

Acknowledgments and Disclosure of Funding

LIAS laboratory, ISAE-ENSMA / Université de Poitiers.

References

Siddharth Bhatia, Mohit Wadhwa, Kenji Kawaguchi, Neil Shah, Philip S. Yu, and Bryan Hooi. Sketch-based anomaly detection in streaming graphs. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 93–104, 2023. doi: 10.1145/3580305.3599504.

Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. Structural temporal graph neural networks for anomaly detection in dynamic graphs. In *ACM International Conference on Information and Knowledge Management*, pages 3747–3756, 2021. doi: 10.1145/3459637.3481955.

Zhenxing Chen, Bo Liu, Meiqing Wang, Peng Dai, Jun Lv, and Liefeng Bo. Generative adversarial attributed network anomaly detection. In *ACM International Conference on Information and Knowledge Management*, pages 1989–1992, 2020.

Kaize Ding, Jundong Li, Rohit Bhanushali, and Huan Liu. Deep anomaly detection on attributed networks. In *SIAM International Conference on Data Mining*, pages 594–602, 2019.

Haoyi Fan, Fengbin Zhang, and Zuoyong Li. AnomalyDAE: Dual autoencoder for anomaly detection on attributed networks. In *IEEE International Conference on Acoustics, Speech, and Signal Processing*, pages 5685–5689, 2020.

Jongha Lee, Sunwoo Kim, and Kijung Shin. SLADE: Detecting dynamic anomalies in edge streams without labels via self-supervised learning. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1506–1517, 2024. doi: 10.1145/3637528.3671845.

Kay Liu, Yingtong Dou, Yue Zhao, Xueying Ding, Xiyang Hu, Ruitong Zhang, Kaize Ding, Canyu Chen, Hao Peng, Kai Shu, et al. BOND: Benchmarking unsupervised outlier node detection on static attributed graphs. In *Advances in Neural Information Processing Systems*, volume 35, pages 27021–27035, 2022.

Kay Liu, Yingtong Dou, Xueying Ding, Xiyang Hu, Ruitong Zhang, Hao Peng, Lichao Sun, and Philip S. Yu. PyGOD: A Python library for graph outlier detection. *Journal of Machine Learning Research*, 25(141):1–9, 2024.

Yixin Liu, Zhao Li, Shirui Pan, Chen Gong, Chuan Zhou, and George Karypis. Anomaly detection on attributed networks via contrastive self-supervised learning. *IEEE Transactions on Neural Networks and Learning Systems*, 2021.

Yixin Liu, Shirui Pan, Yu Guang Wang, Fang Xiong, Liang Wang, Qingfeng Chen, and Vincent CS Lee. Anomaly detection in dynamic graphs via transformer. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12081–12094, 2023. doi: 10.1109/TKDE.2021.3124061.

Shiqi Lou, Qingyue Zhang, Shujie Yang, Yuyang Tian, Zhaoxuan Tan, and Minnan Luo. GADY: Unsupervised anomaly detection on dynamic graphs. *arXiv preprint arXiv:2310.16376*, 2023.

Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z. Sheng, Hui Xiong, and Leman Akoglu. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering*, 2021.

Emaad Manzoor, Sadegh M. Milajerdi, and Leman Akoglu. Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1035–1044, 2016. doi: 10.1145/2939672.2939783.

Jianheng Tang, Fengrui Li, Ziqi Zhao, Yong Li, Rong Wen, Xinlei Zhao, Kay Liu, Xiyang Hu, and Jing Ma. GADBench: Revisiting and benchmarking supervised graph anomaly detection. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Phillip Wenig, Sebastian Schmidl, and Thorsten Papenbrock. TimeEval: A benchmarking toolkit for time series anomaly detection algorithms. *Proceedings of the VLDB Endowment*, 15(12):3678–3681, 2022.

Zhiming Xu, Xiao Huang, Yue Zhao, Yushun Dong, and Jundong Li. Contrastive attributed network anomaly detection with data augmentation. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2022.

Xiao Yang, Xuejiao Zhao, and Zhiqi Shen. A generalizable anomaly detection method in dynamic graphs. In *AAAI Conference on Artificial Intelligence*, 2025. doi: 10.1609/aaai.v39i20.35508.

Wenchao Yu, Wei Cheng, Charu C. Aggarwal, Kai Zhang, Haifeng Chen, and Wei Wang. NetWalk: A flexible deep embedding approach for anomaly detection in dynamic networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2663–2671, 2018. doi: 10.1145/3219819.3220024.

Yue Zhao, Zain Nasrullah, and Zheng Li. PyOD: A Python toolbox for scalable outlier detection. *Journal of Machine Learning Research*, 20(96):1–7, 2019.

Li Zheng, Zhenpeng Li, Jian Li, Zhao Li, and Jun Gao. AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN. In *International Joint Conference on Artificial Intelligence*, pages 4419–4425, 2019.

Appendix A. Comparison with Related Tools

Table 2 compares GraFlag with existing tools for graph anomaly detection and related domains. PyGOD (Liu et al., 2024) is a Python library of detection algorithms with a unified API; its repository also includes the BOND (Liu et al., 2022) benchmark evaluation code. GADBench (Tang et al., 2023) evaluates supervised and unsupervised methods on static graphs. None of these tools offer distributed execution or containerized environments. TimeEval (Wenig et al., 2022) demonstrates the value of containerized, distributed benchmarking for time series but does not support graph data. PyOD (Zhao et al., 2019) targets tabular outlier detection. GraFlag is the first tool to integrate all of these capabilities for graph anomaly detection.

Table 2: Feature comparison of GraFlag with related tools. ✓ = supported, × = not supported, ○ = partial.

Feature	**GraFlag**	**PyGOD**	**GADBench**	**TimeEval**	**PyOD**
Data domain	Graph	Graph	Graph	Time series	Tabular
Dynamic graphs	✓	×	×	—	—
Distributed exec.	✓	×	×	✓	×
Containerized	✓	×	×	✓	×
GPU scheduling	✓	×	×	×	×
Auto. evaluation	✓	○	✓	✓	○
Web dashboard	✓	×	×	○	×
Config replay	✓	×	×	×	×
Plugin metrics	✓	×	×	×	×

Key differentiators include: (*i*) GraFlag is the only graph-focused tool with distributed, containerized execution; (*ii*) nine standardized result types cover three granularities across three temporal modes, while other tools support only node-level scores; (*iii*) the plugin-based metric system allows domain-specific evaluation without modifying framework code; and (*iv*) configuration replay via `service_config.json` enables exact reproduction of any past experiment.

Appendix B. Shared Storage and Project Structure

GraFlag separates the orchestration package from the shared data layer. The main `graflag` package (installed via pip) contains the CLI, Python API, web dashboard, and virtual development cluster. The NFS-mounted shared storage holds all methods, datasets, experiment results, and shared libraries accessible to every cluster node.

Main Package Structure.

```
graflag/
|-- cli.py           # CLI (argparse)
|-- core.py          # GraFlag orchestration class
|-- config.py        # Configuration management
|-- models.py        # Dataclasses (MethodInfo, etc.)
|-- ssh.py           # SSH/rsync operations
|-- docker_ops.py    # Docker Swarm operations
|-- api.py           # Python API (for GUI)
|-- gui/             # Flask web dashboard
`-- devcluster/      # Virtual cluster (Docker Compose)
```

NFS Shared Storage Layout.

```
/shared/                          # NFS mount point
|-- methods/                      # Detection methods
|   `-- <method_name>/
|       |-- .env                  # Metadata + parameters
|       |-- Dockerfile
|       `-- [train_graflag.py]    # Custom methods only
|-- datasets/
|   `-- <dataset_name>/           # Graph data files
|-- experiments/
|   `-- exp__<method>__<dataset>__<timestamp>/
|       |-- results.json          # Scores + ground truth
|       |-- service_config.json   # Reproducible config
|       |-- service_details.json  # Docker service info
|       |-- training.csv          # Training curves
|       `-- eval/                 # Evaluation outputs
|           |-- evaluation.json
|           |-- roc_curve.png
|           |-- pr_curve.png
|           `-- score_distribution.png
`-- libs/
    |-- graflag_runner/           # Execution framework
    |-- graflag_bond/             # PyGOD adapter
    |-- graflag_evaluator/        # Evaluation framework
    `-- graflag_data/             # Dataset metadata + on-demand fetcher
```

Appendix C. Method Integration Guide

GraFlag supports two integration patterns. This section provides complete examples for each.

C.1 Pattern A: Custom Methods

For a custom method, two files are needed: a `.env` configuration, a `Dockerfile`, and require an entry point `COMMAND` which is specified by the `.env` configuration. Defining a new method, needs usually three steps:

Step 1: Method configuration (`.env`). The `.env` file defines method metadata and default parameters. Parameters prefixed with an underscore are injected as environment variables into the container. When the container entry point uses the `--pass-env-args` flag, these variables are converted to command-line arguments (e.g., `_MAX_EPOCH=200` becomes `--max_epoch 200`). The `COMMAND` field specifies the executable invoked inside the container.

The optional `SUPPORTED_DATASETS` field lists comma-separated patterns used to filter compatible datasets; wildcards are expanded to regular expressions (e.g., `bond_*` matches `bond_.*`).

```
METHOD_NAME=taddy
DESCRIPTION=Temporal Anomaly Detection via Transformer
SOURCE_CODE=https://github.com/yixingwang0517/TADDY_pytorch
SUPPORTED_DATASETS=uci,btc_alpha,btc_otc,digg
COMMAND=python3 train_graflag.py

# Default parameters (injected as env vars)
_MAX_EPOCH=200
_LEARNING_RATE=0.001
_EMBEDDING_DIM=32
_NUM_HIDDEN_LAYERS=2
_NUM_ATTENTION_HEADS=2
_WINDOW_SIZE=2
_GPU=0
```

Step 2: Container definition (`Dockerfile`).

```
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
RUN apt-get update && apt-get install -y python3 python3-pip git
RUN git clone <source_repo> /app/src
COPY train_graflag.py /app/
RUN pip install ./libs/graflag_runner torch torch-geometric
WORKDIR /app
CMD ["python3", "-m", "graflag_runner", "--pass-env-args"]
```

The `graflag_runner` module serves as the container entry point. It reads `COMMAND` from the environment, monitors resource usage (CPU, memory, GPU), executes the command, and captures output. Shared libraries (`graflag_runner`, `graflag_bond`) are installed inside the container via the Dockerfile, since the build context is the entire shared storage directory.

Step 3: Integration script. The script referenced by `COMMAND` uses the `ResultWriter` API to produce standardized outputs. Since `graflag_runner` is installed inside the container (Step 2), it can be imported directly:

```python
import os, argparse
from graflag_runner import ResultWriter

parser = argparse.ArgumentParser()
parser.add_argument("--max_epoch", type=int)
parser.add_argument("--learning_rate", type=float)
# ... additional parameters
args = parser.parse_args()

writer = ResultWriter()  # reads EXP env var

# Load data from $DATA env var
data = load_data(os.environ["DATA"])

# Training loop
for epoch in range(args.max_epoch):
    loss = train_step(model, data)
    writer.spot("training", epoch=epoch, loss=loss)

# Save standardized results
scores = model.predict(data)
writer.save_scores(
    result_type="EDGE_STREAM_ANOMALY_SCORES",
    scores=scores,
    ground_truth=data.labels)
writer.add_resource_metrics(
    exec_time_ms=elapsed, peak_memory_mb=mem)
writer.finalize()
```

C.2 Pattern B: PyGOD via graflag_bond

For methods available in the PyGOD library (Liu et al., 2024), only a `.env` and a `Dockerfile` are needed. The `graflag_bond` adapter handles data loading, model instantiation, training, and result writing automatically.

```
# .env for bond_dominant
METHOD_NAME=bond_dominant
DESCRIPTION=Deep Anomaly Detection on Attributed Networks
SUPPORTED_DATASETS=bond_*
COMMAND=python3 -m graflag_bond.train

_HID_DIM=64
_NUM_LAYERS=4
_DROPOUT=0
_LR=0.004
_EPOCH=100
_CONTAMINATION=0.1
```

The `graflag_bond.train` module dynamically resolves the detector class from the method name (e.g., `bond_dominant` → `pygod.detector.Dominant`), extracts all `_`-prefixed environment variables, performs type conversion using the detector's constructor signature, trains the model, and saves `NODE_ANOMALY_SCORES` via `ResultWriter`.

Parameter Override at Runtime. Users can override any default parameter without modifying the `.env` file:

```
graflag run -m bond_dominant -d bond_inj_cora \
    --params EPOCH=200 LR=0.001 HID_DIM=128
```

Appendix D. Supported Datasets

GraFlag currently includes 34 benchmark datasets organized into two categories, as shown in Table 3. Static datasets (14) are sourced from the BOND benchmark (Liu et al., 2022) and comprise six synthetically generated graphs, five real-world networks, and three networks with injected anomalies. Dynamic datasets (20) cover financial transaction networks, communication networks, and social networks, with various temporal representations (snapshots, continuous edge streams).

On-demand fetching via `graflag_data`. No dataset payload is committed to the repository. Each `datasets/<name>/` directory ships only a `metadata.json` file that declares the dataset's original source, the files it needs (with `url`, optional `extract`: `gunzip`/`zip`/`tar`/`tar.gz`/`tar.bz2`, optional `sha256`, optional archive `members` selector), and—when applicable—a `build` step that regenerates derived artifacts from a base dataset (e.g. the `*_snapshot` variants call `convert_to_strgnn.py`). The `graflag_data` library reads this metadata and hydrates missing files on demand: `graflag run` invokes it on the swarm manager via SSH so downloads land directly on the NFS share and are visible to every worker container. HTTP, GitHub raw, and archive sources are handled natively; Google-Drive folders (used by the GeneralDyG preprocessed CSVs) are supported via `gdown` as an optional extra. Users can also prefetch datasets explicitly with `graflag-data fetch <name>` or inspect readiness with `graflag-data status`.

Appendix E. Evaluation System

E.1 Computed Metrics

The `graflag_evaluator` computes the following metrics for all nine result types.

- **AUC-ROC**: Area under the Receiver Operating Characteristic curve, measuring discrimination ability across all thresholds.
- **AUC-PR**: Area under the Precision-Recall curve, more informative than AUC-ROC under class imbalance.
- **Precision@K**, **Recall@K**, **F1@K**: Precision, recall, and F1 score when predicting the top K scored entities as anomalies, where K equals the true number of anomalies.
- **Best F1**: Maximum F1 score across all decision thresholds with the corresponding threshold value.

Table 3: Benchmark datasets in GraFlag. Datasets are organized by graph type (static vs. dynamic) and data source. Additional datasets can be added by placing graph files in the shared `datasets/` directory.

Category	Count	Datasets
Static Graphs		
BOND synthetic	6	bond_gen_{100, 500, 1K, 5K, 10K, time}
BOND injected	3	bond_inj_{amazon, cora, flickr}
BOND real-world	5	bond_{books, disney, enron, reddit, weibo}
Dynamic Graphs		
Financial	6	btc_alpha(_snapshot), btc_otc(_snapshot), generaldyg_btc_{alpha, otc}
Communication	5	uci(_snapshot), email_snapshot(_005), gady_email_dnc
Social	1	digg
Intrusion	2	anograph_{darpa, iscx}
Streaming	6	slade_{bitcoinalpha, bitcoinotc, hijack, wikipedia, new}, streamspot_all

Additional type-specific metrics are computed: temporal span and timestamp count for temporal/streaming types; unique edge and node counts for edge-level types.

E.2 Custom Metric Plugins

Custom metrics can be registered for any result type via a plugin system. The Python API method `register_metric()` extracts the function source and writes it as a plugin file on the cluster. The evaluator loads plugins from two directories:

- **Global**: `libs/graflag_evaluator/plugins/` (all evaluations).
- **Per-experiment**: `experiments/<exp>/custom_metrics/`.

Each plugin file imports `MetricCalculator` and calls `register_metric()` at module level. The plugin file resides on NFS-mounted shared storage, so the containerized evaluator can load it at runtime. Plugins can also be created manually.

E.3 Generated Visualizations

The evaluator produces three standard plots per experiment, saved as PNG files in the `eval/` subdirectory:

- **ROC Curve**
- **Precision-Recall Curve**
- **Score Distribution**

Additionally, for each CSV file created by `ResultWriter.spot()` during training (e.g., `training.csv`), the evaluator generates training curve plots showing metric evolution over epochs. These plots are visible in the web dashboard (Figure 2).

Appendix F. Web Dashboard and Virtual Development Cluster

F.1 Web Dashboard

The web dashboard (`graflag gui`) provides a browser-based interface for experiment management built with Flask and Flask-SocketIO (Figure 2). Key features include:

- **Cluster Overview**: Real-time node status, GPU availability, and running services.
- **Experiment Management**: Browse methods and datasets, configure and submit experiments, monitor progress with live log streaming via WebSocket.
- **One-Click Evaluation**: Trigger evaluation and view metrics and plots directly in the browser.
- **Experiment History**: List past experiments with status badges, access results and evaluation outputs.

The dashboard communicates with the cluster through the same `GraFlag` Python API used by the CLI, ensuring consistent behavior. Background threads poll for state changes and push updates to connected clients via WebSocket, avoiding page reloads. All long-running operations (build, run, evaluate) execute in background threads to keep the interface responsive.

F.2 Virtual Development Cluster

GraFlag includes a virtual development cluster (`graflag devcluster`) that replicates the full distributed environment locally using Docker Compose. This allows users to develop and test methods, debug configurations, and explore the framework without access to physical GPU infrastructure.

The virtual cluster creates Docker-in-Docker containers (one manager and workers) on a custom bridge network with fixed IPs, NFS-based shared storage between containers, and SSH connectivity. It mirrors the production architecture: the manager runs a Docker Swarm, a local registry, and an NFS server, while workers join the swarm and mount the shared directory. The cluster is deployed with a single command:

```
graflag devcluster --hosts hosts.yml
```

where `hosts.yml` specifies the network subnet, manager IP, and worker IPs. GPU passthrough is supported when NVIDIA drivers are available on the host. The cluster is torn down with `graflag devcluster --down`.

Appendix G. CLI Reference

Table 4 lists all GraFlag CLI commands.

Datasets are hydrated automatically by `graflag run` via the `graflag_data` library (Appendix D). The same library ships a companion CLI, invoked directly as `graflag-data <list|status|fetch>`, for manual prefetch, readiness checks, or forced re-downloads.

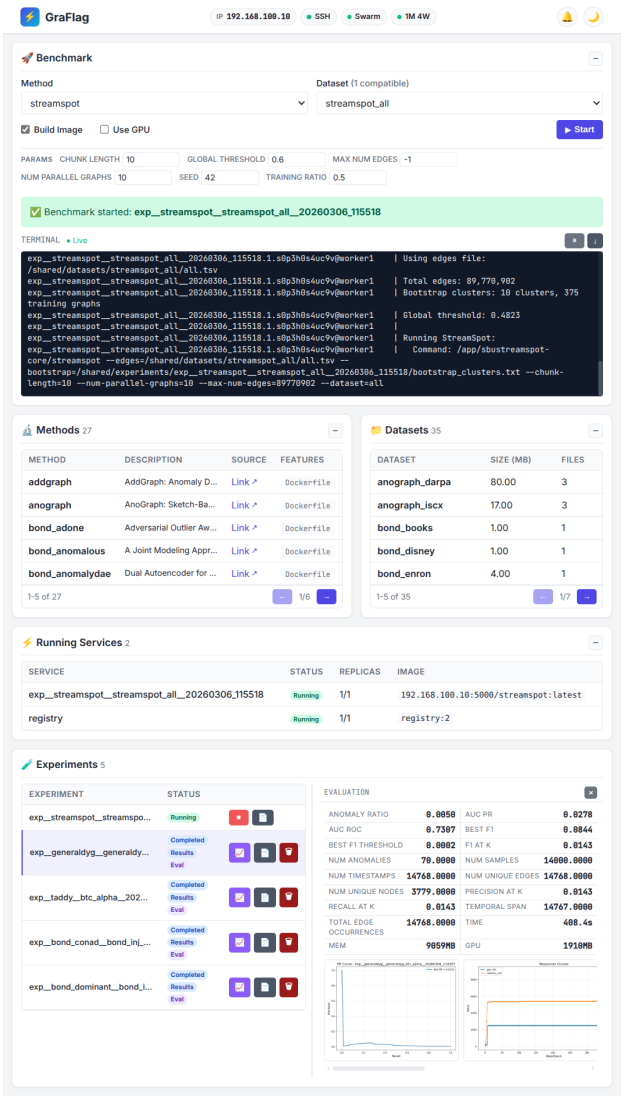

Figure 2: GraFlag web dashboard showing the experiment monitoring interface with real-time log streaming and cluster status.

Table 4: GraFlag CLI commands. All commands are invoked as `graflag <command>`.

Command	Description
`setup`	Initialize Docker Swarm on manager and join workers
`run`	Execute a method on a dataset (`-m`, `-d`, `--build`, `--params`, `--from-config`)
`evaluate`	Compute metrics and generate plots for an experiment (`-e`)
`list`	List resources: `methods`, `datasets`, `experiments`, `services`
`status`	Show cluster status (nodes, services, shared storage)
`logs`	View experiment logs (`-e`, `--follow`, `--tee`)
`stop`	Stop a running experiment (`-e`, `--rm`)
`copy`	Transfer files to/from cluster (`--from-remote`, `-r`)
`sync`	Sync local method or library to remote (`--lib`)
`gui`	Start web dashboard (`--port`, `--debug`)
`devcluster`	Deploy/teardown virtual cluster (`--hosts`, `--down`)